

# Rozpoznawanie pytań na podstawie analizy głosu

Sprawozdanie projektowe

Kacper Jurek, Mateusz Łopaciński

22 czerwca 2025

## Spis treści

|   |                             |   |
|---|-----------------------------|---|
| 1 | Cel pracy                   | 2 |
| 2 | Opis danych                 | 2 |
| 3 | Ekstrakcja cech             | 3 |
| 4 | Tuning hiperparametrów      | 4 |
| 5 | Finalna architektura modelu | 4 |
| 6 | Trening modelu              | 5 |
| 7 | Wyniki                      | 5 |
| 8 | Podsumowanie                | 6 |

## 1 Cel pracy

Celem projektu było opracowanie klasyfikatora, który dla krótkiej próbki audio decyduje, czy wypowiedź została zrealizowana jako **pytanie**, czy **stwierdzenie**. Zadanie potraktowaliśmy jako dwuklasowy problem uczenia nadzorowanego.

### Wstępne koncepcje

1. **Analiza intonacji** – wyłącznie cechy akustyczne (log-mel-spektrogramy), zakładając że pytania w języku polskim mają charakterystyczny akcent w końcowej części zdania.
2. **Analiza składniowa** – wariant hybrydowy: transkrypcja mowy (ASR) → klasyfikator tekstowy (BERT) wykrywający konstrukcje pytań.

### Decyzja ostateczna

Wstępne testy z modelem konwolucyjno-rekurencyjnym (CRNN) operującym na spektrogramach dały satysfakcjonujące wyniki, dlatego w dalszych pracach skoncentrowaliśmy się na dopracowaniu tego podejścia.

## 2 Opis danych

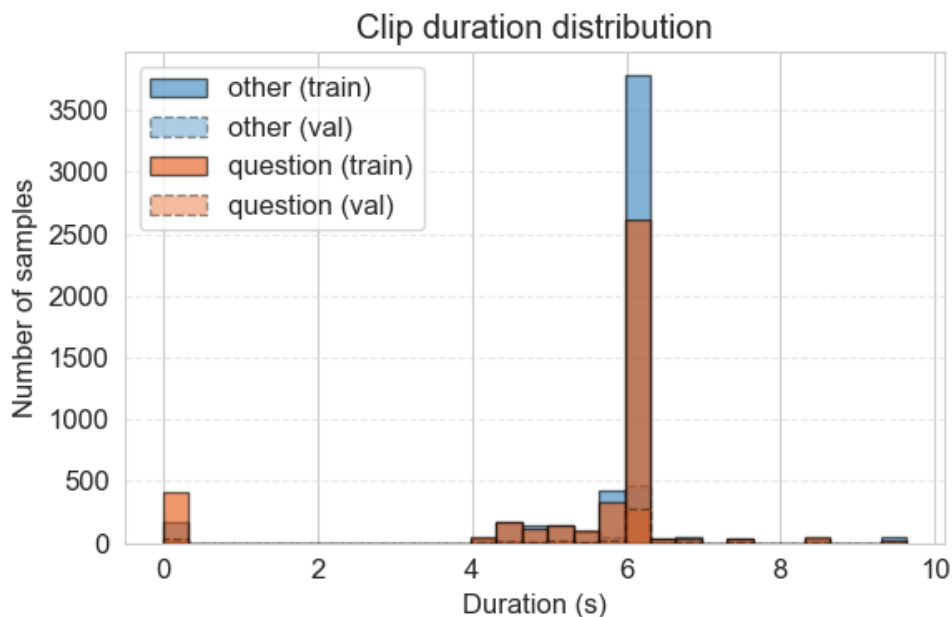
Nagrania pochodzą z audiobooków i zostały ręcznie otagowane etykietami *question/other*. Tabela 1 przedstawia podział na zbiory.

Tabela 1: Rozkład klas w zbiorach danych

| Zbiór | <i>other</i> | <i>question</i> |
|-------|--------------|-----------------|
| train | 5 768        | 4 504           |
| test  | 6 042        | 1 103           |

### Charakterystyka nagrań

- **Kanał:** mono, 16 kHz
- **Długość klipów:** od  $\sim 0.2$  s do 10 s (moda ok. 6 s), rys. 1



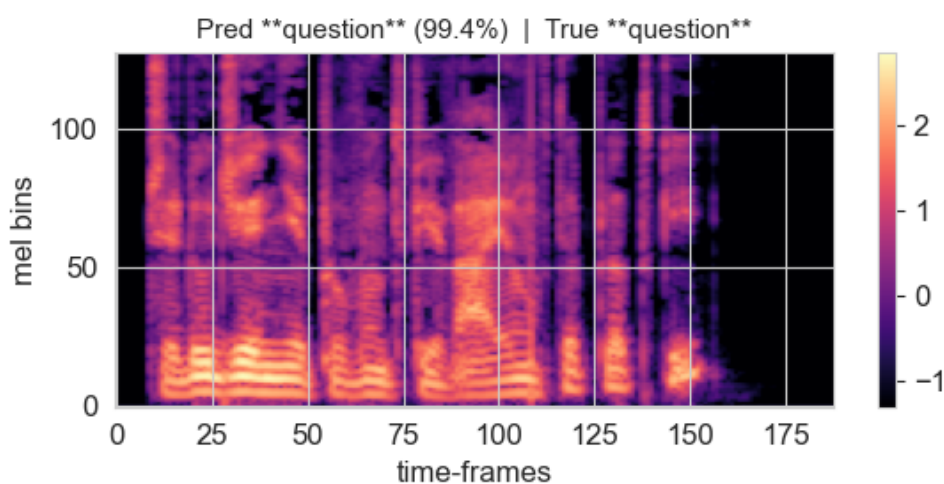
Rysunek 1: Rozkład długości klipów audio.

### 3 Ekstrakcja cech

Z każdego klipu wyliczany jest log-mel-spektrogram:

- 16 kHz, 128 pasm Mel,
- okno FFT = 1024 próbek (64 ms), przesunięcie = 256 próbek (16 ms).

Podczas treningu stosujemy augmentację: maskowanie czasowe (do 30 ms) oraz częstotliwościowe (do 8 pasm).



Rysunek 2: Przykładowy log-mel-spektrogram pytania.

## 4 Tuning hiperparametrów

Optymalizacji dokonano przy użyciu Optuna. Każdy *trial* obejmował szybki trening (6 epok) i ocenę F1 na walidacji (10 % danych treningowych). Łącznie przebadano 20 konfiguracji z MedianPruner.

### Przestrzeń wyszukiwań

- learning\_rate: log-uniform  $[10^{-4}, 10^{-2}]$
- weight\_decay: log-uniform  $[10^{-6}, 10^{-3}]$
- dropout:  $[0.0, 0.5]$
- lstm\_hidden: {96, 128, 160}
- lstm\_layers: {1, 2, 3}

### Najlepsze hiperparametry

```
{'conv1': 48, 'conv2': 96, 'lstm_hidden': 128, 'lstm_layers': 2,  
 'dropout': 0.1, 'lr':  $1.079 \times 10^{-3}$ , 'weight_decay':  $1.057 \times 10^{-4}$ }
```

## 5 Finalna architektura modelu

Listing 1: Finalna implementacja CRNN

```
1 import torch
2 import torch.nn as nn
3
4 class CRNN(nn.Module):
5     def __init__(self):
6         super().__init__()
7
8         # --- Blok CNN ---
9         self.cnn = nn.Sequential(
10             nn.Conv2d(1, 48, kernel_size=3, padding=1),
11             nn.BatchNorm2d(48),
12             nn.ReLU(),
13             nn.MaxPool2d(2),
14             nn.Conv2d(48, 96, kernel_size=3, padding=1),
15             nn.BatchNorm2d(96),
16             nn.ReLU(),
17             nn.MaxPool2d(2),
18         )
19
20         # Obliczenie wymiaru wejścia do LSTM
21         with torch.no_grad():
22             dummy = torch.zeros(1, 1, N_MELS, 100)
23             _, c, f, _ = self.cnn(dummy).shape # c=96
24
```

```

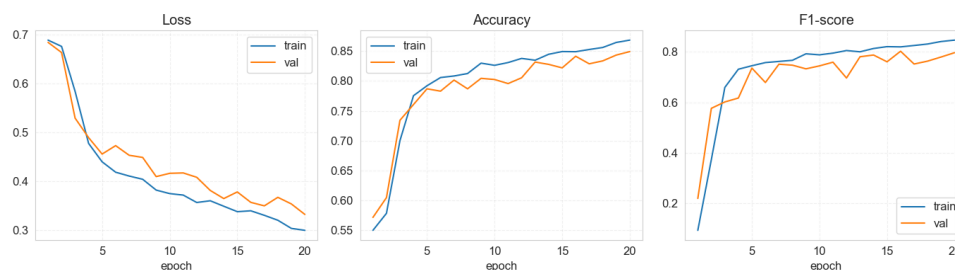
25     # --- BiLSTM ---
26     self.lstm = nn.LSTM(
27         input_size = 96 * f,
28         hidden_size = 128,
29         num_layers = 2,
30         batch_first = True,
31         bidirectional= True,
32         dropout = 0.1
33     )
34
35     # --- Klasyfikator ---
36     self.fc = nn.Linear(256, 2)
37
38     def forward(self, x: torch.Tensor) -> torch.Tensor:
39         x = self.cnn(x.unsqueeze(1)) # (B, 96, F, T)
40         b, c, f, t = x.shape
41         x = x.reshape(b, c * f, t).permute(0, 2, 1) # (B, T, feat)
42         x, _ = self.lstm(x)
43         return self.fc(x[:, -1])

```

Liczba parametrów:  $\approx 3.7$  M.

## 6 Trening modelu

Model trenowano 20 epok (batch = 32) używając optymalizatora Adam ( $1r=1.08 \cdot 10^{-3}$ ,  $weight\_decay=1.06 \cdot 10^{-4}$ ) oraz `torch.cuda.amp` do uczenia w mieszanej precyzji.

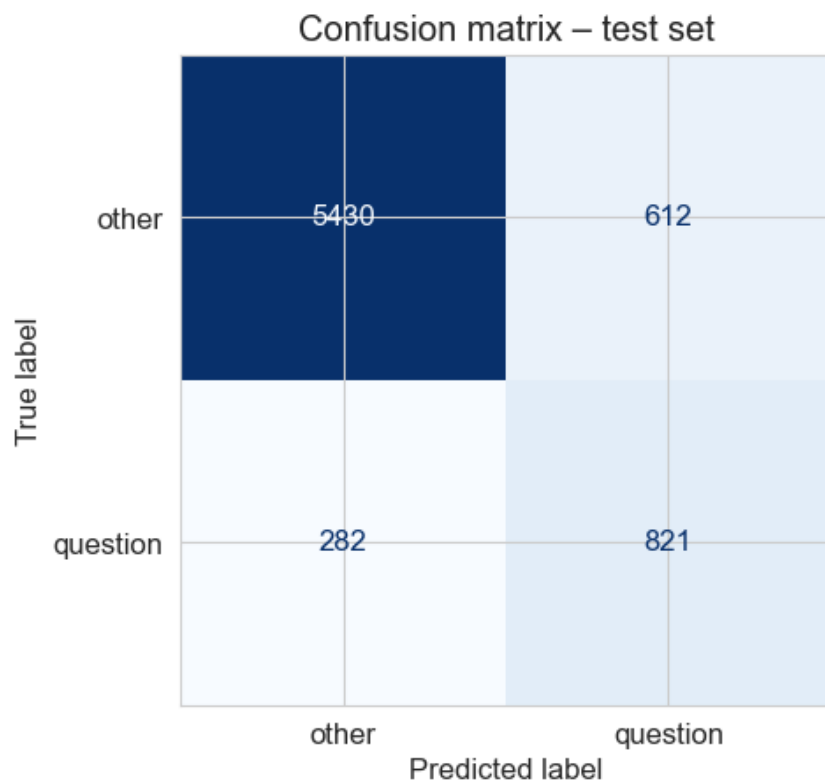


Rysunek 3: Strata, precyzja oraz F1 w funkcji epoki (zbiór treningowy/walidacyjny).

## 7 Wyniki

Tabela 2: Metryki na zbiorze walidacyjnym

| Klasa    | Precision | Recall | F1   | Liczność |
|----------|-----------|--------|------|----------|
| other    | 0.95      | 0.90   | 0.92 | 6 042    |
| question | 0.57      | 0.74   | 0.65 | 1 103    |
| accuracy | 0.87      |        |      |          |



Rysunek 4: Macierz pomyłek – zbiór testowy.

## 8 Podsumowanie

- Końcowy model CRNN osiągnął dokładność 87 % na zbiorze testowym.
- Największą trudność stanowi recall dla klasy *question*; poprawa może wymagać bogatszego zbioru lub silniejszej augmentacji.
- Pomimo uzyskania zadowalających wyników, wciąż istnieje pole do poprawy. Można by było spróbować skorzystać z innej metody lub dodać analizę semantyczną wypowiedzi, w celu zmniejszenia liczby nieprawidłowych klasyfikacji.